Performance Evaluation for Sorting Algorithms

n= 最一開始會排序的總數

range=資料的最大值(均為正整數)

times=每次會執行的次數。每次執行時 n 會多加 n 個

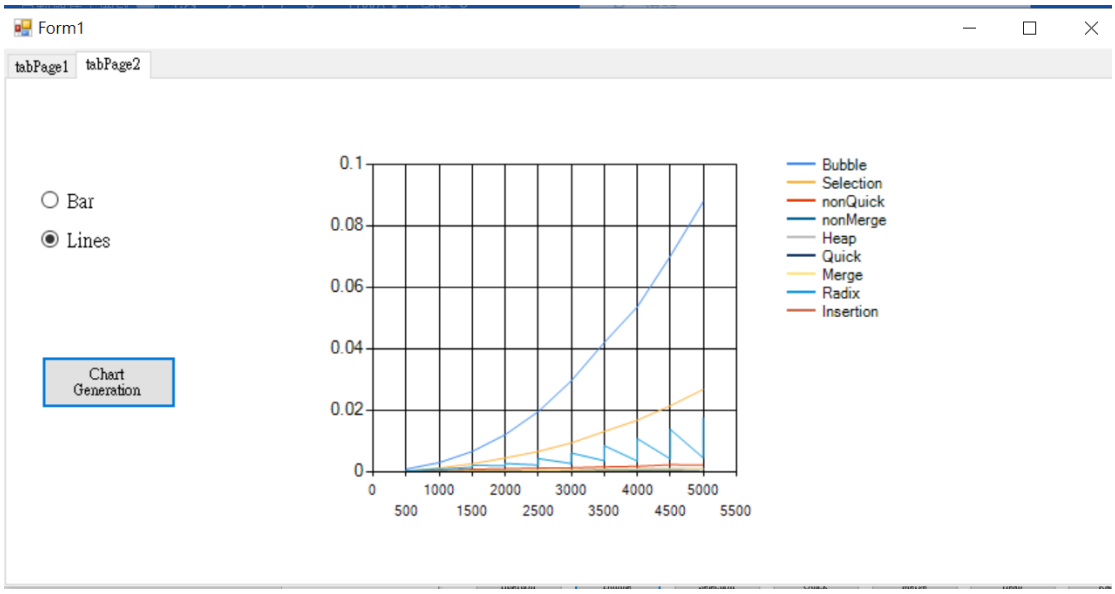
Data Condition Confirm :用來讀取 textbox 中的數據，按下後，才能執行其他按鈕

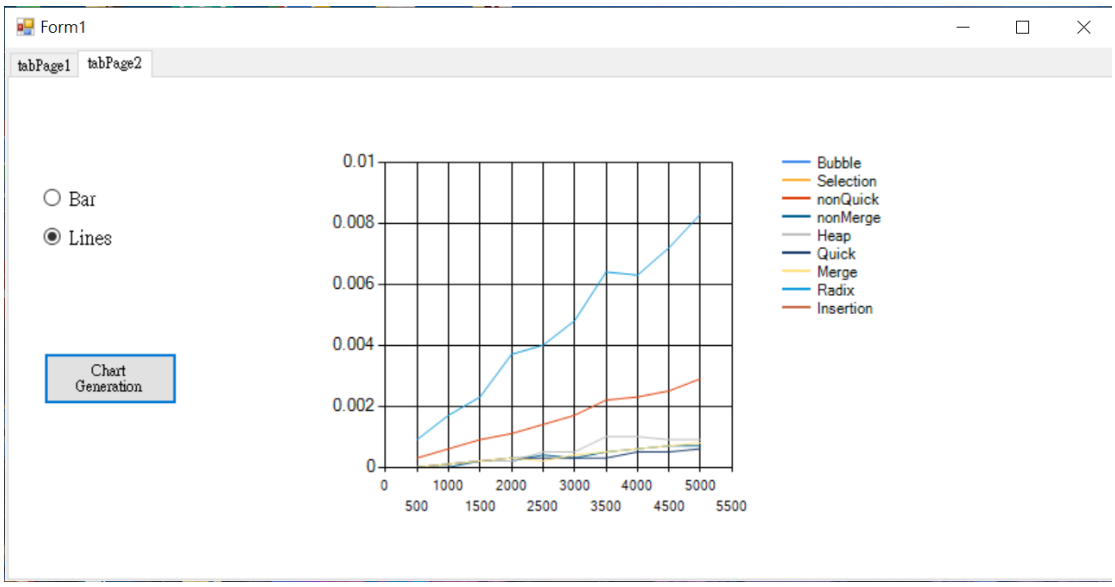
按下各個 sorting 鍵後，每次只要 n 發生變化，就會重新隨機產生亂數資料

Echo print-顯示產生的亂數資料及排序後的結果

時間從長到短:

Bubble->Selection->Radix->Insertion->Heap->Quick/Merge





各排序法原理說明:

Selection Sort:

1. 找到當前範圍中最小值所在位置
2. 將最小值與當前範圍的第一項進行交換

時間複雜度: $O(n^2)$

Bubble Sort:

1. 先將 i 設定為 arr[n-1]，並將 arr[i]設為每次排序的最終項
2. 固定用第 j-1 項與第 j 項比較，如果第 j-1 項大於第 j 項則兩項交換

時間複雜度: $O(n^2)$

Quick Sort:

1.設定左界與右界，只要左界小於右界就進行排序

2.將所有比排序目標(x)小的數字與所有比排序目標(x)大的數字，分別放至(x)的左和右側

3.利用遞迴或迴圈排序所有子範圍內的數字

時間複雜度: O(nlogn)

Insertion Sort:

1.設定 arr[i]為插入陣列中的值

2.將每個比 arr[i]大的值都向後移動一格

3.將 arr[i]插入空位

時間複雜度: $O(n^2)$

Merge Sort:

1.將陣列利用迴圈或遞迴不斷拆分成兩半

2.把每個拆開的陣列依照大小順序合併

3.把合併好的陣列放回原始陣列中

時間複雜度: O(nlogn)

Heap Sort:

1.將資料放入最小堆積中

2.每次放入或拿出資料的時候都用 restore()函式將最小堆積重新排序

3.把資料全部拿出並依序放到原始陣列中

時間複雜度: O(nlogn)

Radix Sort:

1.從最小位數開始

2.把所有資料依照位值順序重新放回原始陣列中

3.不斷操作直到已經排完最大位數

時間複雜度: O(n)